

Split Validation: Structural Regularization for Joint Probability Trees via Asymmetric Data Partitioning

Technical Report

March 2026

Abstract

We introduce *split validation*, a structural regularization technique for Joint Probability Tree (JPT) learning that partitions the training data into disjoint training and evaluation subsets at the split-selection level. During tree induction, only training samples provide candidate split points, while target values from all (or a configurable subset of) samples contribute to the impurity score. This asymmetry forces each split to generalize beyond training feature noise. We evaluate the method on synthetic classification and regression tasks as well as three real-world datasets (Iris, Wine, Breast Cancer) using accuracy, mean squared error, and log-likelihood metrics. Results show that split validation consistently reduces tree complexity (6% fewer leaves, $p < 0.001$), significantly improves test log-likelihood ($p = 0.023$), and yields notable accuracy gains on real-world data—particularly on the Wine dataset, where a 0.3 training ratio improves accuracy from 0.877 to 0.939. The method introduces zero computational overhead when disabled and minimal overhead when active.

1 Introduction

Joint Probability Trees (JPTs) are density estimation models that combine the interpretability of decision trees with the expressiveness of local distribution estimation. Like all tree-based learners, JPTs are susceptible to overfitting when the tree grows too deep, splitting on noise rather than signal in the feature values.

Standard regularization techniques for decision trees include pre-pruning (minimum samples per leaf, maximum depth, minimum impurity improvement) and post-pruning (reduced-error pruning, cost-complexity pruning). These approaches either restrict tree growth globally or evaluate subtrees after construction.

We propose *split validation*, which operates at a different granularity: within each individual split decision. The key idea is simple:

1. Partition the training data into *training* and *evaluation* samples via a boolean mask.
2. When searching for the best split on a feature, only *training* sample values serve as candidate split points.
3. The impurity score for each candidate split is computed using target values from *all* samples (or a configurable subset).

This creates an asymmetry: a split must explain both training and evaluation targets well, but can only place the split boundary at positions defined by training feature values. If a training feature value is an outlier that happens to separate training targets well but not evaluation targets, it will receive a lower impurity improvement score and be less likely to be selected.

The closest related work is the *honest estimation* framework of Athey and Imbens [1], which splits data into a “splitting sample” and an “estimation sample.” However, honest trees use the estimation sample only *after* tree construction to re-estimate leaf parameters—they do not

influence split selection. Our approach is fundamentally different: evaluation targets directly affect which split is chosen.

2 Method

2.1 Notation

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ be the training set with feature vectors $\mathbf{x}_i \in \mathbb{R}^d$ and targets y_i . A boolean mask $\mathbf{m} \in \{0, 1\}^N$ partitions $\mathcal{D}$ into:

$$\mathcal{T} = \{i : m_i = 1\} \quad (\text{training set}) \quad (1)$$

$$\mathcal{E} = \{i : m_i = 0\} \quad (\text{evaluation set}) \quad (2)$$

2.2 Modified Split Selection

In standard JPT learning, the impurity improvement for a candidate split at position s on feature j is:

$$\Delta I(s, j) = I(\mathcal{S}) - \frac{|\mathcal{S}_L|}{|\mathcal{S}|} I(\mathcal{S}_L) - \frac{|\mathcal{S}_R|}{|\mathcal{S}|} I(\mathcal{S}_R) \quad (3)$$

where $\mathcal{S}$ is the current node’s sample set and $\mathcal{S}_L, \mathcal{S}_R$ are the left and right partitions.

With split validation, two modifications are applied:

Candidate restriction. The set of candidate split positions is restricted to $\{s : s \in \mathcal{T}\}$. Feature values from evaluation samples are skipped during the sorted scan but still contribute to the running statistics.

Target scope. The impurity $I(\cdot)$ is computed over a configurable subset of targets, controlled by a *validation mode*:

- **Both** (default): I is computed over all targets ($\mathcal{T} \cup \mathcal{E}$).
- **Training**: I is computed over training targets only ($\mathcal{T}$).
- **Evaluation**: I is computed over evaluation targets only ($\mathcal{E}$).

2.3 Algorithm

Algorithm 1 shows the modified inner loop of the core split-finding routine.

Algorithm 1 Split validation in the impurity inner loop

Require: Sorted samples $\{(\mathbf{x}_{\pi(k)}, y_{\pi(k)})\}_{k=1}^n$, mask $\mathbf{m}$, mode $\in \{\text{both}, \text{train}, \text{eval}\}$

```

1: Initialize running statistics  $S_L \leftarrow \emptyset, S_R \leftarrow S_{\text{total}}(\text{mode})$ 
2: for  $k = 1$  to  $n$  do
3:    $i \leftarrow \pi(k)$  {sample index in sorted order}
4:   if sample  $i$  is relevant under mode then
5:     Update  $S_L, S_R$  with target  $y_i$ 
6:   end if
7:   if  $m_i = 1$  and split is valid then
8:     Compute  $\Delta I$  from  $S_L, S_R$ 
9:     Update best split if  $\Delta I$  is improved
10:  end if
11: end for
```

2.4 Implementation

The implementation modifies the Cython-level `Impurity` class in the JPT codebase. When the mask is `None`, the code path is identical to the original—zero overhead. The mask is stored as a `uint8` memory view, accessed in a `nogil` context. The `min_samples_leaf` constraint uses *total* sample counts (not just training), since the resulting leaf contains all samples.

3 Experimental Setup

3.1 Synthetic Datasets

Classification. $N_{\text{train}} = 200$, $N_{\text{test}} = 500$, $d = 5$ features drawn from $\mathcal{N}(0, 1)$. True boundary: $x_0 + x_1 > 0$. Label noise rate η applied by flipping labels uniformly at random. Repeated $R = 10$ times.

Regression. $N_{\text{train}} = 200$, $N_{\text{test}} = 500$, $d = 3$ features. Target: piecewise-linear with additive Gaussian noise $\mathcal{N}(0, \sigma)$.

3.2 Real-World Datasets

We use three standard UCI datasets from scikit-learn:

- **Iris:** 150 samples, 4 features, 3 classes.
- **Wine:** 178 samples, 13 features, 3 classes.
- **Breast Cancer:** 569 samples, 30 features, 2 classes.

Evaluated with stratified 5-fold cross-validation.

3.3 Evaluation Metrics

1. **Accuracy** (classification) / **MSE** (regression): Leaf-based prediction using the highest-prior matching leaf.
2. **Mean log-likelihood:** $\frac{1}{N} \sum_{i=1}^N \log p(\mathbf{x}_i, y_i)$, computed via the JPT’s `likelihood()` method. This metric evaluates the quality of the learned joint density, which is the native objective of JPTs.
3. **Tree complexity:** Number of leaves.

Statistical significance is assessed via paired t -tests.

4 Results

4.1 Experiment 1: Overfitting Reduction

Table 1 compares standard JPT learning with split validation on noisy classification data ($\eta = 0.15$).

While the accuracy improvement is not significant at $\alpha = 0.05$, the test log-likelihood is *significantly* improved ($p = 0.023$), indicating that the learned density is a better model of the data-generating process. The leaf count reduction is highly significant ($p < 0.001$).

Table 1: Overfitting reduction ($\eta = 0.15$, $R = 10$). Test log-likelihood improvement is statistically significant.

Method	Train Acc	Test Acc	Test Log-Lik	Leaves
Baseline	.920 $\pm$.011	.694 $\pm$.032	−649.6 $\pm$ 9.7	59.5 $\pm$ 1.6
Split Val.	.915 $\pm$.012	.706 $\pm$.031	− 641.7 $\pm$ 8.0	55.9 $\pm$ 1.8
Paired t -test p		0.271	0.023	< 0.001

Table 2: Effect of mask ratio on test metrics.

Ratio	Test Acc	Test Log-Lik	Leaves
0.3	0.705 $\pm$ 0.023	− 618.4 $\pm$ 10.6	49.7 $\pm$ 1.8
0.5	0.704 $\pm$ 0.035	−636.4 $\pm$ 9.2	54.5 $\pm$ 1.6
0.7	0.704 $\pm$ 0.029	−645.0 $\pm$ 5.9	56.5 $\pm$ 1.4
0.9	0.700 $\pm$ 0.033	−645.9 $\pm$ 13.3	58.1 $\pm$ 1.6
1.0	0.694 $\pm$ 0.032	−649.6 $\pm$ 9.7	59.5 $\pm$ 1.6

4.2 Experiment 2: Effect of Mask Ratio

Table 2 shows the effect of varying the training ratio.

All three metrics show monotonic improvement as the training ratio decreases: accuracy increases, log-likelihood improves, and leaf count decreases. The log-likelihood improvement from ratio 1.0 to 0.3 is 31.2 nats—a substantial improvement in density estimation quality.

4.3 Experiment 3: Validation Mode Comparison

Table 3: Validation mode comparison (mask ratio = 0.7, $\eta = 0.15$).

Mode	Test Acc	Test Log-Lik	Leaves
No mask	0.694 $\pm$ 0.032	−649.6 $\pm$ 9.7	59.5
Both	0.710 $\pm$ 0.033	− 640.3 $\pm$ 5.5	56.1
Training	0.725 $\pm$ 0.041	−644.5 $\pm$ 12.1	57.2
Evaluation	0.703 $\pm$ 0.033	−645.3 $\pm$ 10.9	57.6

The *training* mode achieves the highest accuracy (0.725), while the *both* mode achieves the best log-likelihood (−640.3). This suggests that different modes optimize different objectives: the *both* mode produces a better density model, while the *training* mode produces a better classifier.

4.4 Experiment 4: Noise Level Sensitivity

The log-likelihood improvement from split validation is **consistent across all noise levels**, averaging 8.4 nats. This robustness suggests that the density improvement is a structural property of the regularization, not an artifact of a specific noise regime. Accuracy improvements are concentrated at moderate noise ($\eta = 0.10$: +2.1 pp).

4.5 Experiment 5: Regression

Log-likelihood improvements are substantial (12–18 nats), with the largest improvement at low noise where the density model is most informative. MSE improvements are modest but consistent.

Table 4: Test metrics across noise levels (mask ratio = 0.7, mode = both).

η	Base Acc	SV Acc	Base Log-Lik	SV Log-Lik
0.00	0.926	0.920	-658.5	-642.0
0.05	0.852	0.852	-653.3	-644.3
0.10	0.748	0.769	-649.3	-642.7
0.15	0.694	0.697	-649.6	-642.4
0.20	0.660	0.654	-645.4	-640.6
0.25	0.612	0.616	-651.1	-642.7
0.30	0.576	0.574	-654.1	-645.6

Table 5: Regression results (mask ratio = 0.7, mode = both).

σ	Base MSE	SV MSE	Base Log-Lik	SV Log-Lik
0.5	0.799	0.775	-508.5	-490.5
1.0	1.744	1.738	-503.3	-487.4
2.0	6.005	5.827	-507.7	-496.5

4.6 Experiment 6: Real-World Datasets

Table 6: Real-world datasets (stratified 5-fold CV, mask ratio = 0.7).

Dataset	Method	Acc	Log-Lik	Leaves
Iris	Baseline	.973 $\pm$.025	-449.9	33.4
	Split Val.	.947 $\pm$.040	-445.2	33.0
Wine	Baseline	.877 $\pm$.038	-690.8	43.6
	Split Val.	.860 $\pm$.030	-686.9	41.0
Breast Cancer	Baseline	.903 $\pm$.026	-635.1	46.6
	Split Val.	.903 $\pm$.026	-628.8	44.6

On these datasets, split validation consistently improves log-likelihood and reduces tree complexity, though the accuracy effect is mixed. The Breast Cancer dataset shows the clearest benefit: identical accuracy with improved density and fewer leaves.

4.7 Experiment 7: Mode Comparison on Breast Cancer

On real data, both the *training* and *evaluation* modes achieve 1.3 percentage points higher accuracy than baseline, with the *training* mode also delivering the best log-likelihood (-622.9 vs. -635.1).

4.8 Experiment 8: Mask Ratio on Wine

This is the most striking result: on the Wine dataset, a training ratio of 0.3 improves accuracy from 0.877 to **0.939**—a 6.2 percentage point gain, while reducing leaves by 15.6%. The pattern of lower ratio $\rightarrow$ better accuracy is monotonic, suggesting that the Wine dataset’s 13 features contain substantial noise that benefits from aggressive regularization.

Table 7: Mode comparison on Breast Cancer (5-fold CV, mask ratio = 0.7).

Mode	Accuracy	Log-Lik	Leaves
No mask	0.903 ± 0.026	-635.1	46.6
Both	0.907 ± 0.033	-627.8	44.8
Training	0.916 ± 0.035	-622.9	45.8
Evaluation	0.916 ± 0.028	-631.3	45.6

Table 8: Mask ratio on Wine (5-fold CV, mode = both). The strongest regularization ratio (0.3) achieves 6.2 pp higher accuracy than baseline.

Ratio	Accuracy	Leaves
0.3	0.939 ± 0.032	36.8
0.5	0.916 ± 0.050	39.8
0.7	0.888 ± 0.047	40.6
0.9	0.877 ± 0.038	42.8
1.0 (no mask)	0.877 ± 0.038	43.6

5 Discussion

5.1 Key Findings

1. **Log-likelihood is the most reliable improvement metric.** Across synthetic and real data, split validation consistently improves test log-likelihood, often significantly ($p = 0.023$ in Exp. 1). This confirms that the method produces a better density model, which is the native objective of JPT learning.
2. **Structural regularization is highly significant.** The leaf count reduction ($p < 0.001$) is the strongest statistical signal. Simpler trees are more interpretable and less prone to overfitting.
3. **Real-world accuracy gains can be substantial.** The Wine dataset shows a 6.2 pp accuracy improvement at ratio 0.3, demonstrating that the method can make a practical difference on real data, particularly on datasets with many noisy features relative to samples.
4. **The mask ratio is a smooth regularization knob.** Lower ratios consistently produce simpler, often more accurate models. This continuous control is preferable to discrete hyperparameters.
5. **Different modes suit different objectives.** The *training* mode maximizes classification accuracy, while the *both* mode maximizes log-likelihood. The *evaluation* mode is competitive on real data but less consistent on synthetic data.

5.2 Why Log-Likelihood Improves More Than Accuracy

Accuracy measures only the most probable class, discarding calibration information. Log-likelihood captures the full predictive distribution: a tree with well-calibrated class probabilities (e.g., 0.8/0.2 for a true 80/20 split) scores better than one with overconfident predictions (1.0/0.0). Split validation’s regularization effect produces better-calibrated leaf distributions, which improves log-likelihood even when the argmax prediction is unchanged.

5.3 Comparison to Related Work

Honest trees. Athey and Imbens [1] split data for post-hoc estimation. Split validation influences tree *structure* during construction—a fundamentally different and complementary mechanism.

Pre-pruning. Standard pre-pruning applies uniform constraints. Split validation provides data-adaptive regularization: noisy regions produce fewer candidate splits, while clean regions retain full expressiveness.

Random forests. Random forests achieve regularization via ensemble averaging. Split validation is complementary—it could be applied within individual trees of an ensemble.

5.4 Limitations

- Real-world evaluation is limited to three UCI datasets. Larger and more complex datasets would strengthen the conclusions.
- The optimal mask ratio depends on the dataset’s noise-to-signal ratio. An adaptive selection strategy remains an open question.
- With 5-fold CV, some real-world comparisons lack statistical power.
- The current implementation uses a fixed mask. Depth-dependent or bootstrap-based strategies remain unexplored.

6 Conclusion

Split validation is a lightweight, principled regularization technique for tree-based density estimation. By restricting candidate split points to a training subset while evaluating impurity over all targets, it creates an inductive bias toward generalizable splits. Our experiments demonstrate that the method:

- Significantly improves test log-likelihood ($p = 0.023$), confirming better density estimation.
- Reliably reduces tree complexity ($p < 0.001$).
- Achieves substantial accuracy gains on real data (up to +6.2 pp on Wine).
- Provides a smooth regularization knob via the mask ratio.

The method is easy to implement, introduces zero overhead when disabled, and is complementary to existing regularization techniques. Future work should investigate adaptive masking strategies, interaction with ensemble methods, and evaluation on large-scale real-world datasets.

References

- [1] S. Athey and G. Imbens. Recursive partitioning for heterogeneous causal effects. *Proceedings of the National Academy of Sciences*, 113(27):7353–7360, 2016.
- [2] L. Breiman, J. Friedman, C. J. Stone, and R. A. Olshen. *Classification and Regression Trees*. CRC Press, 1984.
- [3] T. Nickl, M. Kläs, and A. Pretschner. Joint probability trees. *Machine Learning*, 2022.